Playwright with Python -- Full Notes

1. What Is Playwright?

Playwright is an open-source browser automation library developed by Microsoft. It allows developers to automate browsers and test web applications using Python and other programming languages.

Playwright supports:

- Chromium
- Firefox
- WebKit

It can automate tasks such as:

- Opening websites.
- Clicking buttons and links.
- Filling out forms.
- Handling dropdowns and checkboxes.
- Taking screenshots.
- Downloading files.
- Handling browser tabs and popups.
- Testing JavaScript-heavy web applications.
- Running end-to-end tests.
- Intercepting and mocking network requests.

2. Why Use Playwright?

Playwright is useful for browser automation and reliable web application testing.

Common use cases:

1. End-to-end testing.
2. UI automation.
3. Form testing.
4. Cross-browser testing.
5. Testing dynamic web pages.
6. API testing.
7. Screenshot testing.
8. Browser-based workflows.

Playwright automatically waits for many elements to become actionable before interacting

with them, reducing the need for manual sleeps.

3. Playwright vs Selenium

Feature	Playwright	Selenium
Browser automation	Yes	Yes
Python support	Yes	Yes
Chromium support	Yes	Yes
Firefox support	Yes	Yes
WebKit support	Yes	Yes
Auto-waiting	Built in for many actions	Often uses explicit waits
Async API	Yes	Selenium Python also supports async through BiDi-related capabilities, but its traditional API is synchronous
API testing	Built-in API request client	Usually uses other libraries or integrations
Network interception	Built-in routing features	Available through browser-specific or BiDi capabilities
Test runner	Playwright Test is available for JavaScript/TypeScript; Python commonly uses pytest	Commonly uses pytest or other test frameworks

Key point: Playwright is often a strong choice for modern browser testing. Selenium remains widely used, especially in established automation frameworks.

4. Install Playwright

Install Playwright for Python:

```javascript.snippet JAVASCRIPT
pip install playwright
```

Install the browser binaries:

```javascript.snippet JAVASCRIPT
playwright install
```

To install only Chromium:

```javascript.snippet JAVASCRIPT
playwright install chromium
```

Install pytest integration:

```javascript.snippet JAVASCRIPT
pip install pytest pytest-playwright
```

Then install the required browser binaries:

```javascript.snippet JAVASCRIPT
playwright install
```

5. Import Playwright

Synchronous API:

```javascript.snippet JAVASCRIPT
from playwright.sync_api import sync_playwright
```

Asynchronous API:

```javascript.snippet JAVASCRIPT
from playwright.async_api import async_playwright
```

Use the synchronous API for simple scripts. Use the asynchronous API when integrating browser automation into an async application.

6. First Playwright Program

```javascript.snippet JAVASCRIPT
from playwright.sync_api import sync_playwrightwith sync_playwright() as p:    browser
=
 p.chromium.launch(headless=False)    page = browser.new_page()    page.goto("https://
www
.python.org")    print("Title:", page.title())    print("URL:", page.url)    browser.
close
()
```

Explanation

- `sync_playwright()` initializes Playwright.
- `p.chromium` selects the Chromium browser engine.
- `launch()` starts the browser.
- `new_page()` creates a new page.
- `goto()` navigates to a URL.
- `page.title()` returns the page title.
- `browser.close()` closes the browser.

`headless=False` displays the browser window. Use `headless=True` or omit the argument for headless execution.

7. Playwright Architecture

The main objects are:

Playwright

Provides access to browser types and other automation capabilities.

Browser

Represents a launched browser process.

BrowserContext

Provides an isolated browser session, similar to a separate browser profile.

Page

Represents a browser tab or page.

Locator

Represents a way to find elements and interact with them.

The typical structure is:

```javascript.snippet JAVASCRIPT
Playwright    |    Browser        |        BrowserContext           |           Page

|               Locator
```

8. Browser Launch

Chromium

```javascript.snippet JAVASCRIPT
browser = p.chromium.launch(headless=False)
```

Firefox

```javascript.snippet JAVASCRIPT
browser = p.firefox.launch(headless=False)
```

WebKit

```javascript.snippet JAVASCRIPT
browser = p.webkit.launch(headless=False)
```

These browser engines require their corresponding Playwright browser binaries to be installed.

9. Navigation Methods

```javascript.snippet JAVASCRIPT
page.goto("https://example.com")
```

Navigate to a URL.

```javascript.snippet JAVASCRIPT
page.go_back()
```

Go back in browser history.

```javascript.snippet
page.go_forward()
```
Go forward in browser history.

```javascript.snippet
page.reload()
```
Reload the current page.

```javascript.snippet
print(page.url)
```
Get the current URL.

```javascript.snippet
print(page.title())
```
Get the page title.

10. Locators in Playwright

Locators identify elements on a web page.

10.1 get_by_role()

Find elements using their accessible role and name.

```javascript.snippet
page.get_by_role("button", name="Login").click()
```
Examples:

```javascript.snippet
page.get_by_role("link", name="About").click()page.get_by_role("textbox", name="Email
").fill(    "
user@example.com")page.get_by_role("checkbox", name="Remember me").check()
```
This is often a good starting point because accessible names make tests easier to understand.

10.2 get_by_text()

```javascript.snippet
page.get_by_text("Welcome").click()
```

For exact text:

```javascript.snippet
page.get_by_text("Welcome", exact=True).click()
```

10.3 get_by_label()

Useful for form fields associated with labels.

```javascript.snippet
page.get_by_label("Email").fill("user@example.com")
```

10.4 get_by_placeholder()

```javascript.snippet
page.get_by_placeholder("Enter your email").fill(    "user@example.com")
```

10.5 get_by_test_id()

```javascript.snippet
page.get_by_test_id("submit-button").click()
```

The default test ID attribute is `data-testid`.

Example HTML:

```javascript.snippet
<button data-testid="submit-button">    Submit</button>
```

10.6 CSS Selector

```javascript.snippet JAVASCRIPT
page.locator("#username").fill("faruk")page.locator(".login-button
").click()page.locator("
input[name='email']").fill(    "user@example.com")
```

10.7 XPath

```javascript.snippet JAVASCRIPT
page.locator("//button[@type='submit']").click()
```

Prefer role, label, placeholder, or test ID locators when appropriate. Use CSS or XPath when they best express the intended target.

11. Locator vs ElementHandle

A `Locator` resolves the element when an action is performed and supports auto-waiting and retry behavior.

```javascript.snippet JAVASCRIPT
button = page.get_by_role("button", name="Login")button.click()
```

An `ElementHandle` refers to a particular DOM element. It is a lower-level API and is generally less suitable for routine interactions.

Best practice: Prefer `Locator` APIs for normal browser automation.

12. Clicking Elements

```javascript.snippet JAVASCRIPT
page.get_by_role("button", name="Submit").click()
```

Double-click:

```javascript.snippet JAVASCRIPT
page.get_by_text("Open file").dblclick()
```

Right-click:

```javascript.snippet JAVASCRIPT
page.locator("#menu").click(button="right")
```

Click with a modifier key:

```javascript.snippet JAVASCRIPT
page.get_by_text("Open file").click(modifiers=["Control"])
```

Playwright waits for the element to satisfy relevant actionability conditions before performing many actions.

13. Filling Input Fields

Fill an Input

```javascript.snippet JAVASCRIPT
page.get_by_label("Username").fill("faruk")
```

Fill a Password

```javascript.snippet JAVASCRIPT
page.get_by_label("Password").fill("example_password")
```

Clear a Field

```javascript.snippet JAVASCRIPT
page.get_by_label("Username").clear()
```

Type Character by Character

```javascript.snippet JAVASCRIPT
page.get_by_label("Username").press_sequentially("faruk")
```

Use `fill()` for ordinary form entry. Use `press_sequentially()` when keyboard-by-keyboard behavior is specifically required.

14. Keyboard Actions

```javascript.snippet JAVASCRIPT
page.keyboard.press("Enter")page.keyboard.press("Escape")page.keyboard.press("Tab")
```

Press a key on a specific element:

```javascript.snippet JAVASCRIPT
page.get_by_label("Search").press("Enter")
```

Keyboard shortcuts:

```javascript.snippet JAVASCRIPT
page.keyboard.press("Control+A")page.keyboard.press("Control+C")page.keyboard.press("
Control
+V")
```

Modifier names and shortcuts can vary by operating system and browser.

15. Getting Text and Attributes

Get Text

```javascript.snippet JAVASCRIPT
text = page.locator("h1").inner_text()print(text)
```

Get Text Content

```javascript.snippet JAVASCRIPT
text = page.locator("h1").text_content()print(text)
```

`inner_text()` returns rendered text, while `text_content()` reads text content from the DOM.

Get an Attribute

```javascript.snippet JAVASCRIPT
href = page.locator("a").first.get_attribute("href")print(href)
```

Get an Input Value

```javascript.snippet JAVASCRIPT
value = page.locator("#username").input_value()print(value)
```

Check Visibility

```javascript.snippet JAVASCRIPT
print(page.locator("#username").is_visible())
```

For test assertions, prefer Playwright's retrying assertions when appropriate rather than immediately checking a value that may not yet be ready.

16. Finding Multiple Elements

Use `locator()` with `count()` or iterate over matching elements.

```javascript.snippet JAVASCRIPT
items = page.locator(".product")print("Count:", items.count())for i in range(items.
count
()):    print(items.nth(i).inner_text())
```

A more compact way:

```javascript.snippet JAVASCRIPT
for item in page.locator(".product").all():    print(item.inner_text())
```

`all()` returns the currently matched locators without waiting for a changing list to stabilize. For dynamic lists, prefer a retrying assertion or wait for a specific condition before collecting them.

17. Dropdowns

For a standard HTML `<select>` element:

```javascript.snippet JAVASCRIPT
page.locator("#country").select_option("bd")
```

Select by label:

```javascript.snippet JAVASCRIPT
page.locator("#country").select_option(    label="Bangladesh")
```

Select by index:

```javascript.snippet JAVASCRIPT
page.locator("#country").select_option(index=0)
```

Get options:

```javascript.snippet JAVASCRIPT
options = page.locator("#country option").all_text_contents()print(options)
```

`select_option()` works with standard HTML select elements. Custom dropdowns usually require clicking the dropdown and selecting an option through normal locators.

18. Checkboxes and Radio Buttons

Check a Checkbox

```javascript.snippet JAVASCRIPT
page.get_by_role(    "checkbox",    name="Remember me").check()
```

Uncheck a Checkbox

```javascript.snippet JAVASCRIPT
page.get_by_role(    "checkbox",    name="Remember me").uncheck()
```

Select a Radio Button

```javascript.snippet JAVASCRIPT
page.get_by_role(    "radio",    name="Male").check()
```

These methods aim to establish the requested state rather than blindly toggling it.

19. Auto-Waiting

Playwright automatically waits for many actions to become possible.

For example:

```javascript.snippet JAVASCRIPT
page.get_by_role(    "button",    name="Submit").click()
```

Before clicking, Playwright checks relevant conditions such as:

- The element is attached to the DOM.
- The element is visible.
- The element is enabled.
- The element can receive pointer events.
- The element is stable for actions that require stability.

This reduces the need for `time.sleep()`.

Avoid arbitrary sleeps:

```javascript.snippet JAVASCRIPT
import timetime.sleep(5)
```

Instead, use locators and explicit conditions.

20. Explicit Waits

Wait for an element to become visible:

```javascript.snippet JAVASCRIPT
page.locator("#result").wait_for(    state="visible",    timeout=10000)
```

Wait for an element to disappear:

```javascript.snippet JAVASCRIPT
page.locator(".loading").wait_for(    state="hidden",    timeout=10000)
```

Wait for a URL:

```javascript.snippet JAVASCRIPT
page.wait_for_url("**/dashboard")
```

Wait for a specific response:

```javascript.snippet JAVASCRIPT
with page.expect_response(    lambda response: "/api/products" in response.url) as
response_info
:    page.get_by_role(        "button",        name="Load products"    ).click()
response
 = response_info.valueprint(response.status)
```

Wait for a response only when the action is expected to trigger that request.

21. Screenshots

Full Page Screenshot

```javascript.snippet JAVASCRIPT
page.screenshot(    path="screenshot.png",    full_page=True)
```

Viewport Screenshot

```javascript.snippet JAVASCRIPT
page.screenshot(path="viewport.png")
```

Screenshot of an Element

```javascript.snippet JAVASCRIPT
page.locator("#profile").screenshot(    path="profile.png")
```

Screenshots are saved relative to the current working directory unless another path is specified.

22. Handling Alerts and Dialogs

Playwright supports JavaScript alerts, confirmations, and prompts.

Accept a dialog:

```javascript.snippet JAVASCRIPT
page.on("dialog", lambda dialog: dialog.accept())
```

Dismiss a dialog:

```javascript.snippet
page.on("dialog", lambda dialog: dialog.dismiss())
```

Read dialog text:

```javascript.snippet
page.on("dialog", lambda dialog: print(dialog.message))
```

For a dialog triggered by an action, register the handler before performing that action:

```javascript.snippet
with page.expect_event("dialog") as dialog_info:    page.get_by_role(        "button
",        name="
Show alert"    ).click()dialog = dialog_info.valueprint(dialog.message)dialog.accept()
```

A dialog that is not handled can block the action that triggered it.

23. Handling New Tabs and Popups

Use `expect_popup()` when an action opens a popup window.

```javascript.snippet
with page.expect_popup() as popup_info:    page.get_by_role(        "link",        name
="
Open new page"    ).click()popup = popup_info.valuepopup.wait_for_load_state()print(
popup
.url)print(popup.title())
```

Use this pattern only when the action is expected to open a new page.

24. Handling Frames and Iframes

Use `frame_locator()` to work inside an iframe.

```javascript.snippet
frame = page.frame_locator("#payment-frame")frame.get_by_label("Card number
").fill(    "
4111111111111111")frame.get_by_role(    "button",    name="Continue").click()
```

The iframe selector must match the actual page.

For most routine interactions, `frame_locator()` is more convenient than manually

switching between frames.

25. File Uploads

For a standard file input:

```javascript.snippet
page.locator(    "input[type='file']").set_input_files("document.pdf")
```

Use an absolute path when needed:

```javascript.snippet
page.locator(    "input[type='file']").set_input_files(    r
"C:\Users\User\Documents\document.pdf"
)
```

Upload multiple files:

```javascript.snippet
page.locator(    "input[type='file']").set_input_files([    "file1.pdf",    "file2.png"
])
```

The files must exist at the specified paths.

26. File Downloads

Use `expect_download()`:

```javascript.snippet
with page.expect_download() as download_info:    page.get_by_role(        "link",

name="Download report"    ).click()download = download_info.valuedownload.save_as("
report
.pdf")
```

This waits for a download triggered by the action and saves it to the specified destination.

27. Browser Contexts

A `BrowserContext` provides an isolated browser session.

```
javascript.snippet JAVASCRIPT

browser = p.chromium.launch()context = browser.new_context()page = context.new_page()
page
.goto("https://example.com")context.close()browser.close()
```

Contexts can isolate cookies, local storage, and other session state.

Create separate contexts when you need independent sessions, such as testing different users.

28. Handling Cookies

Get cookies:

```
javascript.snippet JAVASCRIPT

cookies = context.cookies()print(cookies)
```

Add a cookie:

```
javascript.snippet JAVASCRIPT

context.add_cookies([    {        "name": "theme",        "value": "dark",        "url"
:
 "https://example.com"    }])
```

Clear cookies:

```
javascript.snippet JAVASCRIPT

context.clear_cookies()
```

Use cookies only in authorized test environments and handle session data securely.

29. Scrolling

Scroll an element into view:

```
javascript.snippet JAVASCRIPT

page.locator("#footer").scroll_into_view_if_needed()
```

Scroll using the mouse wheel:

```javascript.snippet JAVASCRIPT
page.mouse.wheel(0, 500)
```

For infinite scrolling, wait for newly loaded content or a change in the result count instead of relying only on a fixed delay.

30. Executing JavaScript

Use `evaluate()` to execute JavaScript in the page.

```javascript.snippet JAVASCRIPT
title = page.evaluate("document.title")print(title)
```

Read the current URL:

```javascript.snippet JAVASCRIPT
url = page.evaluate("window.location.href")
```

Read the page height:

```javascript.snippet JAVASCRIPT
height = page.evaluate(    "document.body.scrollHeight")
```

Use JavaScript when necessary. For normal interactions, prefer Playwright's locator APIs because they better represent user actions and support auto-waiting.

31. Network Interception

Playwright can intercept and modify network requests.

Observe Requests

```javascript.snippet JAVASCRIPT
page.on(    "request",    lambda request: print(        request.method,        request.
url
    ))
```

Observe Responses

```javascript.snippet JAVASCRIPT
page.on(    "response",    lambda response: print(        response.status,
response
.url    ))
```

Abort an Image Request

```javascript.snippet JAVASCRIPT
page.route(    "**/*.{png,jpg,jpeg}",    lambda route: route.abort())
```

Mock an API Response

```javascript.snippet JAVASCRIPT
page.route(    "**/api/user",    lambda route: route.fulfill(        status=200,

content_type="application/json",        body='{"name": "Faruk"}'    ))
```

Network routing is useful for testing error states, offline-like behavior, and deterministic responses.

32. API Testing with Playwright

Playwright for Python includes an API request client.

```javascript.snippet JAVASCRIPT
from playwright.sync_api import sync_playwrightwith sync_playwright() as p:    request
=
 p.request.new_context(        base_url="https://jsonplaceholder.typicode.com
"    )    response = request.get("
/posts/1")    print(response.status)    data = response.json()    print(data["title"])

request.dispose()
```

Use API requests for testing endpoints without launching a browser for every operation.

33. Headless Mode

Headless mode runs the browser without displaying its graphical interface.

```javascript.snippet JAVASCRIPT
browser = p.chromium.launch(headless=True)
```

This is useful for:

- CI/CD pipelines.
- Automated testing servers.
- Background browser tasks.
- Environments without a desktop display.

Test in headed mode when debugging browser interactions.

34. Browser Configuration

Set a viewport:

```javascript.snippet JAVASCRIPT
browser = p.chromium.launch()context = browser.new_context(    viewport={
"width"
: 1280,        "height": 720    })page = context.new_page()
```

Set a locale:

```javascript.snippet JAVASCRIPT
context = browser.new_context(    locale="en-US")
```

Set a timezone:

```javascript.snippet JAVASCRIPT
context = browser.new_context(    timezone_id="Asia/Dhaka")
```

Use explicit configuration when your tests depend on screen size, locale, or timezone.

35. Async Playwright

Playwright also supports asynchronous Python.

```
javascript.snippet                                                    JAVASCRIPT

import asynciofrom playwright.async_api import async_playwrightasync def main():
async
 with async_playwright() as p:        browser = await p.chromium.launch(
headless
=False        )        page = await browser.new_page()        await page.goto("https://
example
.com")        print(await page.title())        await browser.close()asyncio.run(main())
```

In the async API, most browser operations require `await`.

Use the async API when integrating with an asynchronous Python application or when coordinating multiple browser operations concurrently.

36. Playwright with Pytest

Install dependencies:

```
javascript.snippet                                                    JAVASCRIPT

pip install pytest pytest-playwrightplaywright install
```

Create `test_homepage.py`:

```
javascript.snippet                                                    JAVASCRIPT

from playwright.sync_api import Page, expectdef test_homepage_title(page: Page):
page
.goto("https://example.com")    expect(page).to_have_title("Example Domain")
```

Run tests:

```
javascript.snippet                                                    JAVASCRIPT

pytest -v
```

The `page` fixture is provided by the Playwright pytest plugin.

37. Assertions

Playwright provides retrying assertions through `expect()`.

```javascript.snippet JAVASCRIPT
from playwright.sync_api import expectexpect(page).to_have_title("Example Domain
")expect(    page.get_by_role("
heading", name="Example Domain")).to_be_visible()expect(    page.get_by_role("link
", name="
More information")).to_be_visible()
```

These assertions automatically retry until the condition is satisfied or the timeout is reached.

This is generally more reliable than immediately reading a value and comparing it before the page has finished updating.

38. Page Object Model

The Page Object Model (POM) separates page interactions from test cases.

Example: `login_page.py`

```javascript.snippet JAVASCRIPT
from playwright.sync_api import Pageclass LoginPage:    def __init__(self, page: Page):

self.page = page        self.username = page.get_by_label("Username
")        self.password = page.get_by_label("
Password")        self.login_button = page.get_by_role(            "button
",            name="
Login"        )    def open(self):        self.page.goto("https://example.com/login")

def login(self, username, password):        self.username.fill(username)        self.
password
.fill(password)        self.login_button.click()
```

Usage:

```javascript.snippet JAVASCRIPT
login_page = LoginPage(page)login_page.open()login_page.login(    "test_user",
"test_password"
)
```

The selectors and URL must match the application being tested.

39. Complete Login Test Example

```javascript.snippet JAVASCRIPT
from playwright.sync_api import Page, expectdef test_login(page: Page):    page.goto("
https
://example.com/login")    page.get_by_label("Username").fill("test_user
")    page.get_by_label("
Password").fill(        "example_password"    )    page.get_by_role(        "button
",        name="
Login"    ).click()    expect(page).to_have_url(        "https://example.com/dashboard"

)
```

This is an illustrative test. Replace the URL, credentials, and selectors with values from your authorized test application.

40. Common Playwright Errors

TimeoutError

The expected action or condition did not complete before the timeout.

Possible solutions:

- Check the locator.
- Check whether the element exists.
- Use a suitable condition.
- Verify that navigation or network activity completed.
- Inspect the test screenshot or trace.

Strict Mode Violation

A locator matched multiple elements when a single target was expected.

Solution:

```javascript.snippet JAVASCRIPT
page.get_by_role(    "button",    name="Save",    exact=True).click()
```

If multiple matching buttons are legitimate, narrow the locator or deliberately choose `.first` or `.nth(index)`.

Browser Executable Not Found

Install the required browsers:

```javascript.snippet JAVASCRIPT
playwright install
```

Element Not Found

Check the locator and whether the element is inside an iframe, a popup, or a different page.

Navigation Timeout

Check the target URL, network connectivity, and navigation behavior. Use an appropriate wait condition rather than increasing timeouts without understanding the cause.

41. Best Practices

1. Prefer role-based and label-based locators.
2. Use explicit assertions with `expect()`.
3. Avoid unnecessary `time.sleep()` calls.
4. Use isolated browser contexts for separate users.
5. Reuse the pytest `page` fixture when appropriate.
6. Keep page objects focused and reusable.
7. Store credentials securely.
8. Use meaningful test names.
9. Capture screenshots and traces when debugging failures.
10. Use stable test IDs for complex interfaces when needed.
11. Avoid brittle XPath expressions tied to deep DOM structures.
12. Test against staging environments whenever possible.
13. Respect website terms, access controls, rate limits, and privacy.
14. Close browser resources when using Playwright outside managed test fixtures.

42. Selenium vs Playwright: When to Choose

Choose **Playwright** when:

- You are starting a new Python browser-testing project.
- You need strong auto-waiting behavior.
- You want convenient network interception.
- You need isolated browser contexts.
- You want modern end-to-end testing tools.

Choose **Selenium** when:

- Your existing project already uses Selenium.

- Your organization has established Selenium infrastructure.
- You depend on a particular Selenium-supported integration.
- Your team's existing automation framework is built around WebDriver.

Both tools are capable. The best choice depends on your project and testing environment.

43. Practice Projects

Beginner:

1. Open a website and print its title.
2. Fill out a search form.
3. Click a button and verify the result.
4. Take a full-page screenshot.
5. Extract links from a page.

Intermediate:

1. Automate a login test.
2. Handle a popup and an iframe.
3. Upload a file.
4. Download a report.
5. Mock an API response.

Advanced:

1. Create a Page Object Model.
2. Build a Pytest test suite.
3. Run tests against Chromium, Firefox, and WebKit.
4. Generate screenshots and traces for failures.
5. Build an end-to-end test suite for a Django application.

44. Quick Revision Cheat Sheet

javascript.snippet JAVASCRIPT

```
# Browser setupfrom playwright.sync_api import sync_playwright, expectwith sync_playwright() as p:
```

Conclusion

Playwright is a powerful tool for browser automation and end-to-end testing with Python.

Focus on these topics in order:

1. Installation and browser setup.
2. Locators and interactions.

3. Auto-waiting and assertions.
4. Forms, dropdowns, frames, and popups.
5. Screenshots, downloads, and network interception.
6. API testing and asynchronous programming.
7. Pytest and Page Object Model.
8. Real-world testing projects.

Official resources:

- Python documentation: https://playwright.dev/python/
- Python API reference: https://playwright.dev/python/docs/api/class-playwright
- Pytest plugin: https://playwright.dev/python/docs/test-runners